

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

An ancient civilization called **Xylos** protected its interstellar systems using DNA-based locks. Each lock accepts a DNA string **S**, which contains only the characters **A, T, C, G**, and always has an even length between 6 and 50. To open the lock, the string must be decoded correctly.

The decoding is done **two characters (A block) at a time**, from left to right.

- **First character of each block**
 1. Normally, apply the complement rule: $A \leftrightarrow T, C \leftrightarrow G$
 2. If the previous decoded character was G, apply rotation instead: $A \rightarrow G, C \rightarrow T, G \rightarrow A, T \rightarrow C$ (Trap rule)
- **Second character of each block**
 1. Normally, apply rotation: $A \rightarrow G, C \rightarrow T, G \rightarrow A, T \rightarrow C$
 2. If the first decoded character of the same block is A, apply the complement instead.
- Append the decoded characters in order to form the final string.

Print the final decoded DNA string.

Input	Output
ATCGAT	TCGATC
GATACA	CGCGTG

Test Case 2 Explanation:

Block 1 (G A):

First character G $\rightarrow$ Complement Rule $\rightarrow$ C (prev = none, normal complement)

Second character A $\rightarrow$ Rotation Rule $\rightarrow$ G

Result so far: CG

Block 2 (T A):

Previous decoded character = G $\rightarrow$ Trap Rule applies, first character uses Rotation instead of Complement $T \rightarrow C$

Second character A $\rightarrow$ Rotation Rule $\rightarrow$ G (first decoded $\neq$ A, normal rotation)

Result so far: CGCG

Block 3 (C A):

Previous decoded character = G $\rightarrow$ Trap Rule applies, first character uses Rotation $C \rightarrow T$

Second character A (first decoded = T $\rightarrow$ normal rotation) $\rightarrow$ G

Final decoded string: CGCGTG

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

A lost civilization called **Zyron** secured their space stations using DNA-based locks. Each lock takes a DNA string **S**, which contains only the letters **A**, **T**, **C**, **G** and always has an even length between 6 and 50. To unlock a station, the string must be decoded correctly.

The decoding is done **two characters at a time** from left to right.

- **First character of each block:**
 1. Normally, apply the complement rule: $A \leftrightarrow T$, $C \leftrightarrow G$
 2. If the previous decoded character was **G**, apply rotation instead:
 $A \rightarrow G$, $C \rightarrow T$, $G \rightarrow A$, $T \rightarrow C$ (Trap Rule)
- **Second character of each block:**
 1. Normally, apply rotation: $A \rightarrow G$, $C \rightarrow T$, $G \rightarrow A$, $T \rightarrow C$
 2. If the first decoded character of the same block is **A**, apply complement instead (Mirror Rule)

Print the fully decoded DNA string

Input	Output
TACGGA	ATGACG

Explanation:

Block 1 (T A):

First character T $\rightarrow$ Complement Rule $\rightarrow$ A (prev = none, normal complement)

Second character A $\rightarrow$ Mirror Rule applies (first decoded = A) $\rightarrow$ T

Result so far: AT

Block 2 (C G):

Previous decoded character = T $\rightarrow$ Complement Rule $\rightarrow$ C $\rightarrow$ G

Second character G $\rightarrow$ Rotation Rule $\rightarrow$ A (first decoded $\neq$ A, normal rotation)

Result so far: ATGA

Block 3 (G A):

Previous decoded character = A $\rightarrow$ Complement Rule $\rightarrow$ G $\rightarrow$ C

Second character A $\rightarrow$ Rotation Rule $\rightarrow$ G (first decoded $\neq$ A, normal rotation)

Final decoded string: ATGACG

CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics

In case of any circumstance not present in the rubrics, feel free to discuss in assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

[SET-A]

SOLUTION

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();

        String result = "";
        char prevDecoded = ' ';
        int i = 0;

        while (i < s.length()) {

            // First character
            char first = s.charAt(i);
            char decodedFirst;

            if (prevDecoded == 'G') { // rotation
                if (first == 'A') decodedFirst = 'G';
                else if (first == 'C') decodedFirst = 'T';
                else if (first == 'G') decodedFirst = 'A';
                else decodedFirst = 'C'; // T
            } else { // complement
                if (first == 'A') decodedFirst = 'T';
                else if (first == 'T') decodedFirst = 'A';
                else if (first == 'C') decodedFirst = 'G';
                else decodedFirst = 'C'; // G
            }

            result = result + decodedFirst;

            // Second character
            char second = s.charAt(i + 1);
            char decodedSecond;

            if (decodedFirst == 'A') { // complement
                if (second == 'A') decodedSecond = 'T';
                else if (second == 'T') decodedSecond = 'A';
                else if (second == 'C') decodedSecond = 'G';
                else decodedSecond = 'C'; // G
            } else { // rotation
                if (second == 'A') decodedSecond = 'G';
                else if (second == 'C') decodedSecond = 'T';
                else if (second == 'G') decodedSecond = 'A';
                else decodedSecond = 'C'; // T
            }

            result = result + decodedSecond;
            prevDecoded = decodedSecond;

            i = i + 2;
        }
        System.out.println(result);
    }
}
```

Rubric For Set A and Set B

SL	Points To Meet	Marks [10]
1	Correctly providing input and output and other things	2
2	Correctly implementing the complement rule for the first char	2
3	Correctly implementing the trap rule for the first char	2
4	Correctly implementing the rotation rule for the second char	2
6	Correctly implementing the complement rule for the second char	2

[SET-B]

SOLUTION

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        String result = "";
        char prevDecoded = ' ';
        int i = 0;

        while (i < s.length()) {

            // First character
            char first = s.charAt(i);
            char decodedFirst;

            if (prevDecoded == 'G') { // Trap rule: rotation instead of complement
                if (first == 'A') decodedFirst = 'G';
                else if (first == 'C') decodedFirst = 'T';
                else if (first == 'G') decodedFirst = 'A';
                else decodedFirst = 'C'; // T
            } else { // Normal complement
                if (first == 'A') decodedFirst = 'T';
                else if (first == 'T') decodedFirst = 'A';
                else if (first == 'C') decodedFirst = 'G';
                else decodedFirst = 'C'; // G
            }

            result = result + decodedFirst;

            // Second character
            char second = s.charAt(i + 1);
            char decodedSecond;

            if (decodedFirst == 'A') { // Mirror rule: complement instead of rotation
                if (second == 'A') decodedSecond = 'T';
                else if (second == 'T') decodedSecond = 'A';
                else if (second == 'C') decodedSecond = 'G';
                else decodedSecond = 'C'; // G
            } else { // Normal rotation
                if (second == 'A') decodedSecond = 'G';
                else if (second == 'C') decodedSecond = 'T';
                else if (second == 'G') decodedSecond = 'A';
                else decodedSecond = 'C'; // T
            }

            result = result + decodedSecond;
            prevDecoded = decodedSecond;

            i = i + 2;
        }

        System.out.println(result);
    }
}
```


Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

A farmer is having difficulty organizing fruits into baskets, each with a fixed capacity. Write a Java program that should take the **number of fruit types** as input, store **their quantities** in an integer array and display the initial basket capacities. Your task is to **assign these fruits to the baskets** by following the fruit allocation rules.

Fruit Allocation Rules:

- A basket can only hold fruits if its capacity is **equal to or greater than** the number of fruits to be placed. Otherwise, the basket is skipped.
- After placing fruits into a basket, the basket's capacity is **reduced** by the number of fruits allocated.

Finally, the program should display **updated capacities** of the basket and the **fruit types that could not be placed** in any basket, along with their corresponding quantities.

Given basket capacities : `basket_capacity[] = {5, 6, 7, 2, 3}`

Sample Input	Sample Output
Enter the number of fruit types: 3 Enter the quantity of each fruit type: For Fruit type 1 : 6 For Fruit type 2 : 9 For Fruit type 3 : 2	Initial basket capacities: 5 6 7 2 3 Fruit type 2 with quantity 9 could not be placed Updated basket capacities: 3 0 7 2 3
Explanation : • Fruit type 1 (6 fruits) is placed in the second basket (capacity 6). After allocation, the basket capacity is reduced to 0. • Fruit type 2 (9 fruits) cannot be placed in any basket because no basket has enough capacity, so it remains unplaced. • Fruit type 3 (2 fruits) is placed in the first basket (capacity 5). After allocation, the basket capacity is reduced to 3.	

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

A teacher records the attendance of students using their roll numbers, but due to manual entry, the roll numbers are not arranged in a proper order. Write a Java program that should accept the **number of students present**, store their roll numbers in an **integer array** and display the **original attendance list**. After that, the program must **rearrange the roll numbers** so that **even numbers appear first followed by the odd numbers** by maintaining the **original order**. Finally, the program should **display the reordered attendance list**, along with the **counts for even roll numbers and odd numbers**, the **highest and lowest roll numbers**.

Constraints:

- The reordered list must be out-of-place.
- You cannot use any built-in function other than `.length`.

Sample Input	Sample Output
Enter number of students present: 8 Enter roll numbers: Student 1: 15 Student 2: 8 Student 3: 21 Student 4: 4 Student 5: 10 Student 6: 7 Student 7: 18 Student 8: 3	Original attendance list: 15 8 21 4 10 7 18 3 Reordered attendance list: 8 4 10 18 15 21 7 3 Number of even roll numbers: 4 Number of odd roll numbers: 4 Highest roll number: 21 Lowest roll number: 3
Explanation : In the sample input, the teacher enters 8 students and their roll numbers: 15, 8, 21, 4, 10, 7, 18, and 3. The program first displays these as the original attendance list. It then rearranges the roll numbers by placing all even numbers first (8, 4, 10, 18) followed by all odd numbers (15, 21, 7, 3), while keeping their original order. The program counts 4 even and 4 odd roll numbers. Finally, it identifies 21 as the highest roll number and 3 as the lowest roll number.	

CSE110 Fall 2025 Lab Exam - 04 Tentative Solutions and Rubrics

In case of any circumstance not present in the rubrics, feel free to discuss in assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

[SET-A]

SOLUTION

```
import java.util.Scanner;

public class LabExam4SetA{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] basket_capacity = {5, 6, 7, 2, 3};

        System.out.print("Enter the number of fruit types: ");
        int n = sc.nextInt();
        int[] fruits = new int[n];

        System.out.println("Enter the quantity of each fruit
type:");
        for (int i = 0; i < n; i++) {
            System.out.print("For Fruit type " + (i + 1) + " :
");
            fruits[i] = sc.nextInt();
        }

        System.out.println("Initial basket capacities:");
        for (int c = 0; c < basket_capacity.length; c++ ) {
            System.out.print(basket_capacity[c] + " ");
        }
        System.out.println();

        for (int i = 0; i < n; i++) {
            int fruit = fruits[i];
            boolean placed = false;

            for (int j = 0; j < basket_capacity.length; j++) {
                if (basket_capacity[j] >= fruit) {
                    basket_capacity[j] -= fruit;
                    placed = true;
                    break;
                }
            }

            if (!placed) {
                System.out.println("Fruit type " + (i + 1) + "
```

CSE110 Fall 2025 Lab Exam - 04 Tentative Solutions and Rubrics

```
with quantity " + fruit + " could not be placed.");
    }
}

System.out.println("Updated basket capacities:");
for (int c = 0; c < basket_capacity.length; c++) {
    System.out.print(basket_capacity[c] + " ");
}
}
```

RUBRIC

SL	Criteria	Marks
1	Correctly takes the number of fruit types as input, creates the array and reads the quantity of each fruit type into an array.	2
2	Prints the initial basket capacities correctly before allocation.	0.5
3	Accurately assigns each fruit type to the first basket with sufficient capacity; skips baskets that cannot accommodate the fruit.	3
4	Correctly reduces the basket capacity by the number of fruits allocated.	1.5
5	Identifies and displays fruit types that could not be placed, along with their quantities.	2
6	Prints the updated capacities of all baskets after allocation.	1

[SET-B]

SOLUTION

```
import java.util.Scanner;

public class LabExam4SetB{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of students present: ");
        int n = sc.nextInt();
        int[] roll = new int[n];
        System.out.println("Enter roll numbers:");
        for (int i = 0; i < n; i++) {
            System.out.print("Student " + (i + 1) + ": ");
            roll[i] = sc.nextInt();
        }
        System.out.println("\nOriginal attendance list:");
        for (int i = 0; i < n; i++) {
            System.out.print(roll[i] + " ");
        }
        System.out.println();
        int evenCount = 0, oddCount = 0;
        int highest = roll[0];
        int lowest = roll[0];
        for (int i = 0; i < n; i++) {
            if (roll[i] % 2 == 0)
                evenCount++;
            else
                oddCount++;
            if (roll[i] > highest)
                highest = roll[i];
            if (roll[i] < lowest)
                lowest = roll[i];
        }
        int[] reordered = new int[n];
        int index = 0;
        for (int i = 0; i < n; i++) {
            if (roll[i] % 2 == 0) {
                reordered[index++] = roll[i];
            }
        }
        for (int i = 0; i < n; i++) {
            if (roll[i] % 2 != 0) {
                reordered[index++] = roll[i];
            }
        }
    }
}
```

CSE110 Fall 2025 Lab Exam - 04 Tentative Solutions and Rubrics

```
    }
    System.out.println("\nReordered attendance list:");
    for (int i = 0; i < n; i++) {
        System.out.print(reordered[i] + " ");
    }
    System.out.println();
    System.out.println("\nNumber of even roll numbers: " +
evenCount);
    System.out.println("Number of odd roll numbers: " +
oddCount);
    System.out.println("\nHighest roll number: " + highest);
    System.out.println("Lowest roll number: " + lowest);
    }
}
```

RUBRIC

SL	Points To Meet	Marks [10]
1	The program correctly accepts the number of students and their roll numbers, storing them in an array.	1
2	Displays the original roll numbers in the order entered.	0.5
3	Correctly counts the number of even and odd roll numbers.	1.5
4	Finds and displays the highest and lowest roll numbers correctly.	2.5
5	Reorders the array so all even numbers appear first, then odd numbers, preserving original order.	4
6	Displays the reordered list correctly.	0.5